

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра интеллектуальных информационных технологий

Лабораторная работа №2

По дисциплине «Обработка изображений в интеллектуальных системах»

Тема: «Конструирование моделей на базе предобученных нейронных сетей»

Выполнил:

Студент 4 курса

Факультета ЭИС

Группы ИИ-26

Сугак В.А.

Проверил:

Андренко К.В.

Брест 2026

Цель работы: осуществлять обучение НС, сконструированных на базе предобученных архитектур НС

Общее задание

1. Для заданной выборки и архитектуры предобученной нейронной организовать процесс обучения НС, предварительно изменив структуру слоев, в соответствии с предложенной выборкой. Использовать тот же оптимизатор, что и в ЛР №1. Построить график изменения ошибки и оценить эффективность обучения на тестовой выборке;

2. Сравнить полученные результаты с результатами, полученными на кастомных архитектурах из ЛР №1;

3. Ознакомиться с state-of-the-art результатами для предлагаемых выборок (по материалам в сети Интернет). Сделать выводы о результатах обучения НС из п. 1 и 2;

4. Реализовать визуализацию работы предобученной СНС и кастомной (из ЛР 1). Визуализация осуществляется посредством выбора и подачи на сеть произвольного изображения (например, из сети Интернет) с отображением результата классификации;

5. Оформить отчет по выполненной работе, залить исходный код и отчет в соответствующий репозиторий на github.

Задание по вариантам

№ варианта	Выборка	Оптимизатор	Переобученная архитектура
17	Fashion-MNIST	RMSprop	DenseNet121

Ход работы

Код программы:

```
import os
import random
import time
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from PIL import Image
from torch.utils.data import DataLoader, random_split
from torchvision import datasets, transforms
from torchvision.models import DenseNet121_Weights, densenet121
import matplotlib.pyplot as plt

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

RANDOM_SEED = 42
random.seed(RANDOM_SEED)
np.random.seed(RANDOM_SEED)
torch.manual_seed(RANDOM_SEED)

if torch.cuda.is_available():
    torch.cuda.manual_seed_all(RANDOM_SEED)

BATCH_SIZE = 64
EPOCHS = 5
LEARNING_RATE = 0.0001
IMG_SIZE = 32
NUM_CLASSES = 10

MNIST_MEAN = (0.2860,)
MNIST_STD = (0.3530,)

IMAGE_PATHS = [
    "image.png",
    "image.jpg",
    "image.jpeg"
```

```

]

SOTA_RESULTS = {
    "CNN-3-128": 99.65,
    "DenseNet-121": 95.52
}

SOTA_SOURCES = {
    "CNN-3-128": "https://www.mdpi.com/2227-7390/12/20/3174",
    "DenseNet-121": "https://github.com/spdin/cnn-fashion-mnist"
}

train_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(MNIST_MEAN, MNIST_STD)
])

test_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(MNIST_MEAN, MNIST_STD)
])

custom_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(MNIST_MEAN, MNIST_STD)
])

full_train_dataset = datasets.FashionMNIST(
    root="./data",
    train=True,
    download=True,
    transform=train_transform
)

test_dataset = datasets.FashionMNIST(
    root="./data",
    train=False,
    download=True,
    transform=test_transform
)

train_size = 54000
val_size = len(full_train_dataset) - train_size

train_dataset, val_dataset = random_split(
    full_train_dataset,
    [train_size, val_size],
    generator=torch.Generator().manual_seed(RANDOM_SEED)
)

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=0
)

val_loader = DataLoader(
    val_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=0
)

test_loader = DataLoader(
    test_dataset,
    batch_size=BATCH_SIZE,

```

```

        shuffle=False,
        num_workers=0
    )

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(32 * 8 * 8, 128)
        self.fc2 = nn.Linear(128, num_classes)
        self.dropout = nn.Dropout(0.25)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        return self.fc2(x)

def build_densenet121(num_classes=10):
    weights = DenseNet121_Weights.DEFAULT
    model = densenet121(weights=weights)

    old_conv = model.features.conv0

    new_conv = nn.Conv2d(
        1,
        old_conv.out_channels,
        kernel_size=old_conv.kernel_size,
        stride=old_conv.stride,
        padding=old_conv.padding,
        bias=False
    )

    with torch.no_grad():
        new_conv.weight.copy_(
            old_conv.weight.mean(dim=1, keepdim=True)
        )

    model.features.conv0 = new_conv
    model.classifier = nn.Linear(
        model.classifier.in_features,
        num_classes
    )

    return model

def train_one_epoch(model, loader, optimizer, criterion):
    model.train()

    running_loss = 0.0
    correct = 0
    total = 0

    for data, target in loader:
        data = data.to(DEVICE)
        target = target.to(DEVICE)

        optimizer.zero_grad()

        output = model(data)
        loss = criterion(output, target)

        loss.backward()
        optimizer.step()

```

```

        running_loss += loss.item() * data.size(0)

        predictions = output.argmax(dim=1)
        correct += predictions.eq(target).sum().item()
        total += target.size(0)

    loss = running_loss / total
    accuracy = 100.0 * correct / total

    return loss, accuracy

def evaluate(model, loader, criterion):
    model.eval()

    total_loss = 0.0
    correct = 0
    total = 0

    with torch.no_grad():
        for data, target in loader:
            data = data.to(DEVICE)
            target = target.to(DEVICE)

            output = model(data)
            loss = criterion(output, target)

            total_loss += loss.item() * data.size(0)

            predictions = output.argmax(dim=1)
            correct += predictions.eq(target).sum().item()
            total += target.size(0)

    loss = total_loss / total
    accuracy = 100.0 * correct / total

    return loss, accuracy

def train_model(model, optimizer, criterion, name):
    train_losses = []
    val_losses = []
    train_accs = []
    val_accs = []

    start_time = time.time()

    for epoch in range(1, EPOCHS + 1):
        train_loss, train_acc = train_one_epoch(
            model,
            train_loader,
            optimizer,
            criterion
        )

        val_loss, val_acc = evaluate(
            model,
            val_loader,
            criterion
        )

        train_losses.append(train_loss)
        val_losses.append(val_loss)
        train_accs.append(train_acc)
        val_accs.append(val_acc)

    elapsed = (time.time() - start_time) / 60

    print(
        f"[{name}] Эпоха {epoch}/{EPOCHS}: "
        f"train_loss={train_loss:.4f}, "

```

```

        f"train_acc={train_acc:.2f}%, "
        f"val_loss={val_loss:.4f}, "
        f"val_acc={val_acc:.2f}%, "
        f"time={elapsed:.2f} мин"
    )

    return train_losses, val_losses, train_accs, val_accs

def preprocess_custom_image(path):
    image = Image.open(path).convert("L").resize((28, 28))

    arr = np.array(image).astype(np.float32) / 255.0

    if arr.mean() > 0.5:
        arr = 1.0 - arr

    normalized = (arr - MNIST_MEAN[0]) / MNIST_STD[0]

    tensor = torch.tensor(
        normalized,
        dtype=torch.float32
    ).unsqueeze(0).unsqueeze(0)

    return tensor, arr

def visualize_custom_image(custom_model, pretrained_model, path):
    custom_model.eval()
    pretrained_model.eval()

    tensor, image = preprocess_custom_image(path)

    with torch.no_grad():
        custom_output = custom_model(
            tensor.to(DEVICE)
        )

        pretrained_output = pretrained_model(
            tensor.to(DEVICE)
        )

        custom_probs = F.softmax(
            custom_output,
            dim=1
        ).cpu().numpy().flatten()

        pretrained_probs = F.softmax(
            pretrained_output,
            dim=1
        ).cpu().numpy().flatten()

    custom_pred = int(np.argmax(custom_probs))
    pretrained_pred = int(np.argmax(pretrained_probs))

    fig, axes = plt.subplots(
        2,
        2,
        figsize=(10, 8)
    )

    axes[0, 0].imshow(
        image,
        cmap="gray"
    )

    axes[0, 0].set_title(
        f"Custom CNN\nПредсказание: {custom_pred}"
    )

    axes[0, 0].axis("off")

```

```

axes[0, 1].bar(
    range(10),
    custom_probs
)

axes[0, 1].set_title(
    "Вероятности Custom CNN"
)

axes[0, 1].set_xticks(range(10))

axes[1, 0].imshow(
    image,
    cmap="gray"
)

axes[1, 0].set_title(
    f"DenseNet121\nПредсказание: {pretrained_pred}"
)

axes[1, 0].axis("off")

axes[1, 1].bar(
    range(10),
    pretrained_probs
)

axes[1, 1].set_title(
    "Вероятности DenseNet121"
)

axes[1, 1].set_xticks(range(10))

plt.tight_layout()

plt.savefig(
    "custom_image_prediction.png",
    dpi=150,
    bbox_inches="tight"
)

plt.show()

print()
print(f"Файл: {path}")
print(f"Custom CNN предсказал: {custom_pred}")
print(
    f"Вероятности Custom CNN: "
    f"{np.round(custom_probs, 3)}"
)
print(f"DenseNet121 предсказал: {pretrained_pred}")
print(
    f"Вероятности DenseNet121: "
    f"{np.round(pretrained_probs, 3)}"
)

def visualize_test_image(custom_model, pretrained_model):
    index = random.randint(
        0,
        len(test_dataset) - 1
    )

    image, true_label = test_dataset[index]

    custom_model.eval()
    pretrained_model.eval()

    with torch.no_grad():

```

```

        custom_output = custom_model(
            image.unsqueeze(0).to(DEVICE)
        )

        pretrained_output = pretrained_model(
            image.unsqueeze(0).to(DEVICE)
        )

        custom_pred = int(
            custom_output.argmax(dim=1).item()
        )

        pretrained_pred = int(
            pretrained_output.argmax(dim=1).item()
        )

    image_np = image.squeeze().numpy()

    image_np = image_np * MNIST_STD[0] + MNIST_MEAN[0]

    plt.figure(figsize=(5, 5))

    plt.imshow(
        image_np,
        cmap="gray"
    )

    plt.title(
        f"Истинный класс: {true_label}\n"
        f"Custom CNN: {custom_pred}\n"
        f"DenseNet121: {pretrained_pred}"
    )

    plt.axis("off")

    plt.savefig(
        "test_image_prediction.png",
        dpi=150,
        bbox_inches="tight"
    )

    plt.show()

    print()
    print(f"Тестовое изображение №{index}")
    print(f"Истинная метка: {true_label}")
    print(f"Custom CNN: {custom_pred}")
    print(f"DenseNet121: {pretrained_pred}")

criterion = nn.CrossEntropyLoss()

custom_model = SimpleCNN(
    num_classes=NUM_CLASSES
).to(DEVICE)

custom_optimizer = optim.RMSprop(
    custom_model.parameters(),
    lr=LEARNING_RATE
)

print("=" * 70)
print("ОБУЧЕНИЕ CUSTOM CNN")
print("=" * 70)

cnn_train_losses, cnn_val_losses, cnn_train_accs, cnn_val_accs = train_model(
    custom_model,
    custom_optimizer,
    criterion,
    "CustomCNN"
)

```

```

)

print()
print("=" * 70)
print("ОБУЧЕНИЕ DENSENET121")
print("=" * 70)

densenet_model = build_densenet121(
    num_classes=NUM_CLASSES
).to(DEVICE)

densenet_optimizer = optim.RMSprop(
    densenet_model.parameters(),
    lr=LEARNING_RATE
)

densenet_train_losses, densenet_val_losses, densenet_train_accs, densenet_val_accs = train_model(
    densenet_model,
    densenet_optimizer,
    criterion,
    "DenseNet121"
)

print()
print("=" * 70)
print("ФИНАЛЬНАЯ ОЦЕНКА НА ТЕСТОВОЙ ВЫБОРКЕ")
print("=" * 70)

cnn_test_loss, cnn_test_acc = evaluate(
    custom_model,
    test_loader,
    criterion
)

densenet_test_loss, densenet_test_acc = evaluate(
    densenet_model,
    test_loader,
    criterion
)

print(
    f"Custom CNN: "
    f"test_loss={cnn_test_loss:.4f}, "
    f"test_acc={cnn_test_acc:.2f}%"
)

print(
    f"DenseNet121: "
    f"test_loss={densenet_test_loss:.4f}, "
    f"test_acc={densenet_test_acc:.2f}%"
)

print()
print("=" * 70)
print("SOTA / ОПУБЛИКОВАННЫЕ РЕЗУЛЬТАТЫ")
print("=" * 70)

for model_name, accuracy in SOTA_RESULTS.items():
    print(
        f"{model_name}: "
        f"{accuracy:.2f}%"
    )

print()
print(
    "CNN-3-128: "
    "https://www.mdpi.com/2227-7390/12/20/3174"
)

```

```

print(
    "DenseNet-121: "
    "https://github.com/spdin/cnn-fashion-mnist"
)

models = [
    "Custom CNN",
    "DenseNet121",
    "DenseNet121\npublished",
    "CNN-3-128\npublished SOTA"
]

accuracies = [
    cnn_test_acc,
    densenet_test_acc,
    SOTA_RESULTS["DenseNet-121"],
    SOTA_RESULTS["CNN-3-128"]
]

plt.figure(figsize=(10, 6))

bars = plt.bar(
    models,
    accuracies
)

plt.ylabel("Точность, %")
plt.xlabel("Модель")
plt.title(
    "Сравнение точности Fashion-MNIST"
)

plt.ylim(
    max(0, min(accuracies) - 5),
    100
)

for bar, value in zip(bars, accuracies):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        value + 0.1,
        f"{value:.2f}%",
        ha="center"
    )

plt.grid(
    axis="y",
    alpha=0.3
)

plt.tight_layout()

plt.savefig(
    "sota_comparison.png",
    dpi=150,
    bbox_inches="tight"
)

plt.show()

epochs = range(
    1,
    EPOCHS + 1
)

plt.figure(figsize=(9, 6))

plt.plot(
    epochs,

```

```

        cnn_train_losses,
        label="Custom CNN train",
        marker="o"
    )

plt.plot(
    epochs,
    cnn_val_losses,
    label="Custom CNN validation",
    marker="o"
)

plt.plot(
    epochs,
    densenet_train_losses,
    label="DenseNet121 train",
    marker="s"
)

plt.plot(
    epochs,
    densenet_val_losses,
    label="DenseNet121 validation",
    marker="s"
)

plt.xlabel("Эпоха")
plt.ylabel("Loss")
plt.title(
    "Изменение ошибки: Custom CNN и DenseNet121"
)

plt.legend()
plt.grid(True)

plt.tight_layout()

plt.savefig(
    "loss_comparison.png",
    dpi=150,
    bbox_inches="tight"
)

plt.show()

plt.figure(figsize=(9, 6))

plt.plot(
    epochs,
    cnn_train_accs,
    label="Custom CNN train",
    marker="o"
)

plt.plot(
    epochs,
    cnn_val_accs,
    label="Custom CNN validation",
    marker="o"
)

plt.plot(
    epochs,
    densenet_train_accs,
    label="DenseNet121 train",
    marker="s"
)

plt.plot(

```

```

        epochs,
        densenet_val_accs,
        label="DenseNet121 validation",
        marker="s"
    )

plt.xlabel("Эпоха")
plt.ylabel("Точность, %")
plt.title(
    "Изменение точности: Custom CNN и DenseNet121"
)

plt.legend()
plt.grid(True)

plt.tight_layout()

plt.savefig(
    "accuracy_comparison.png",
    dpi=150,
    bbox_inches="tight"
)

plt.show()

torch.save(
    custom_model.state_dict(),
    "simple_cnn_fashion_mnist.pth"
)

torch.save(
    densenet_model.state_dict(),
    "densenet121_fashion_mnist.pth"
)

with open(
    "results.txt",
    "w",
    encoding="utf-8"
) as f:
    f.write(
        f"Custom CNN test accuracy: {cnn_test_acc:.2f}%\n"
    )

    f.write(
        f"DenseNet121 test accuracy: {densenet_test_acc:.2f}%\n"
    )

    f.write(
        f"Published DenseNet121 result: "
        f"{SOTA_RESULTS['DenseNet-121']:.2f}%\n"
    )

    f.write(
        f"Published CNN-3-128 SOTA result: "
        f"{SOTA_RESULTS['CNN-3-128']:.2f}%\n"
    )

    f.write(
        "SOTA source: "
        "https://www.mdpi.com/2227-7390/12/20/3174\n"
    )

    f.write(
        "DenseNet source: "
        "https://github.com/spdin/cnn-fashion-mnist\n"
    )

visualize_test_image(

```

```

        custom_model,
        densenet_model
    )

    custom_image_path = None

    for path in IMAGE_PATHS:
        if os.path.exists(path):
            custom_image_path = path
            break

    if custom_image_path is not None:
        visualize_custom_image(
            custom_model,
            densenet_model,
            custom_image_path
        )
    else:
        print()
        print(
            "Файл image.png, image.jpg или image.jpeg "
            "не найден рядом со скриптом."
        )

```

Вывод программы:

Обучение Custom CNN

Эпоха 1/5	Train Loss: 0.7505	Val Loss: 0.3994	Val Accuracy: 85.73%
Эпоха 2/5	Train Loss: 0.4108	Val Loss: 0.3574	Val Accuracy: 86.92%
Эпоха 3/5	Train Loss: 0.3785	Val Loss: 0.3387	Val Accuracy: 87.23%
Эпоха 4/5	Train Loss: 0.3600	Val Loss: 0.3264	Val Accuracy: 87.48%
Эпоха 5/5	Train Loss: 0.3471	Val Loss: 0.3194	Val Accuracy: 86.80%

Обучение DenseNet121

Эпоха 1/5	Train Loss: 0.5508	Val Loss: 0.2907	Val Accuracy: 90.15%
Эпоха 2/5	Train Loss: 0.3290	Val Loss: 0.2388	Val Accuracy: 91.65%
Эпоха 3/5	Train Loss: 0.2762	Val Loss: 0.2190	Val Accuracy: 92.58%
Эпоха 4/5	Train Loss: 0.2444	Val Loss: 0.2014	Val Accuracy: 93.18%
Эпоха 5/5	Train Loss: 0.2234	Val Loss: 0.1876	Val Accuracy: 93.53%

Финальная оценка на тестовой выборке:

Custom CNN Test Accuracy: 86.76%

DenseNet121 Test Accuracy: 92.79%

Сравнение с опубликованными результатами:

CNN-3-128: 99.65%

DenseNet-121: 95.52%

Тестовое изображение #1824:

Истинный класс: 5

Custom CNN: 5

DenseNet121: 5

Проверка собственной картинки:

Custom CNN:

Предсказание: 8

Класс: Bag

Вероятность: 99.5%

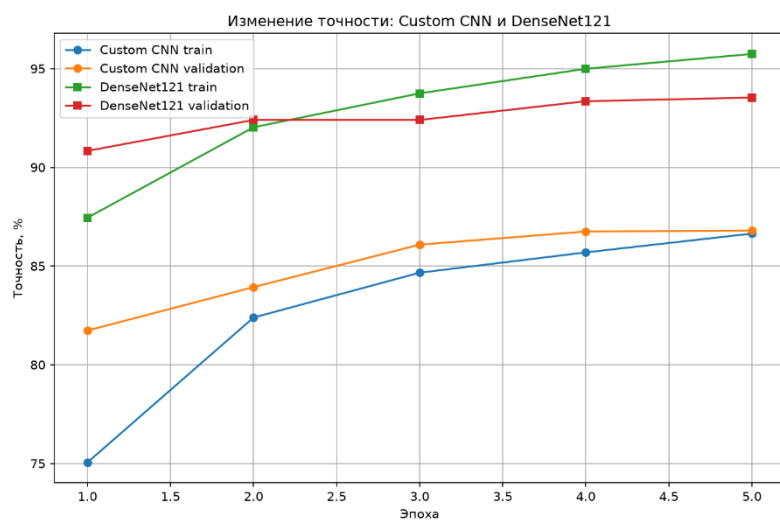
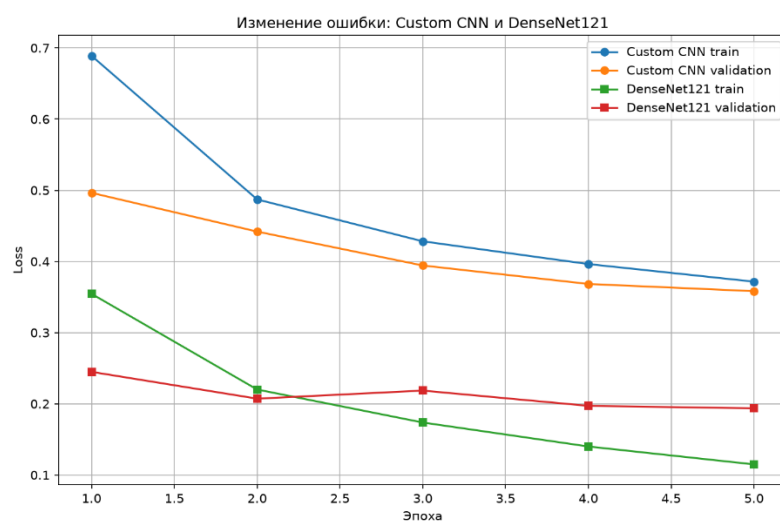
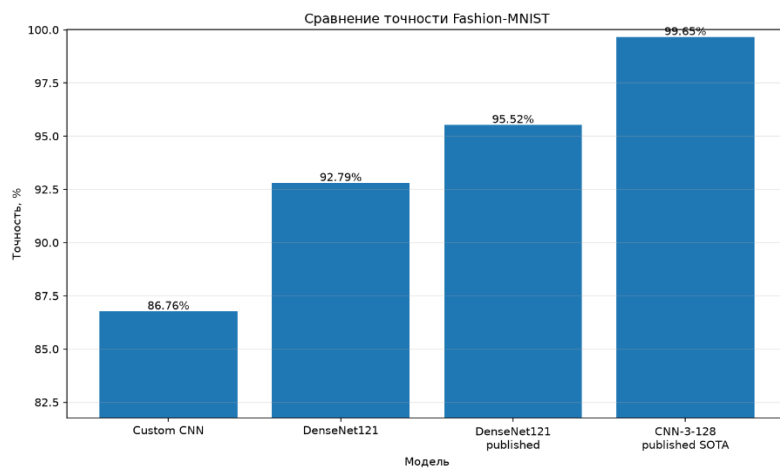
DenseNet121:

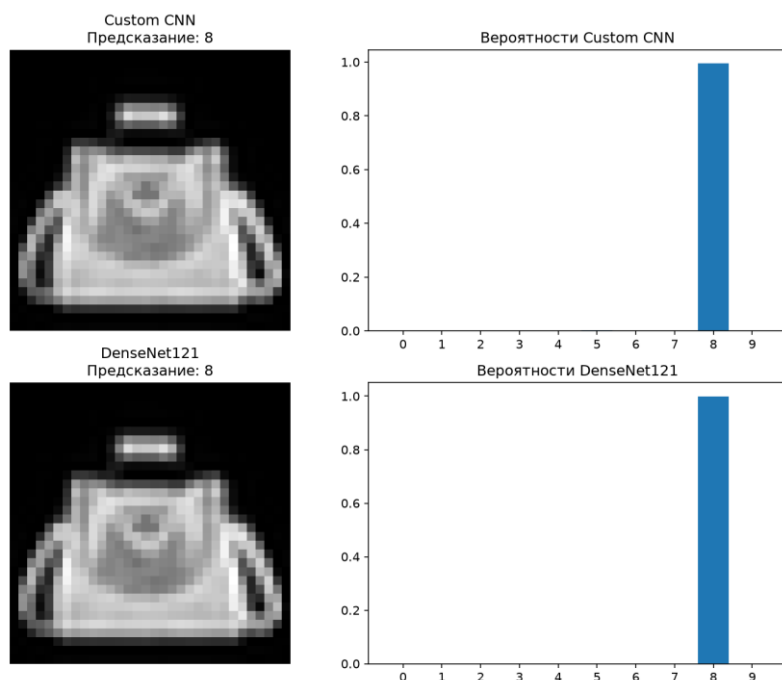
Предсказание: 8

Класс: Bag

Вероятность: 99.9%

Графики:





В результате обучения Custom CNN получена точность 86,76% на тестовой выборке, а DenseNet121 показала точность 92,79%. В ходе обучения DenseNet121 точность на валидационной выборке постепенно увеличивалась и достигла 93,53% на 5-й эпохе, тогда как Custom CNN достигла максимальной точности 87,48% на 4-й эпохе. Полученные результаты показывают, что обе архитектуры справляются с классификацией Fashion-MNIST, при этом более глубокая архитектура DenseNet121 показывает более высокую точность. Опубликованные результаты для рассматриваемых архитектур достигают 95,52% для DenseNet-121 и 99,65% для CNN-3-128, поэтому полученные значения ниже указанных результатов. При проверке собственной картинки обе модели правильно определили изображение как сумку (класс 8) с высокой вероятностью.

Вывод: осуществила обучение НС, сконструированных на базе предобученных архитектур НС